Domain-Driven Design

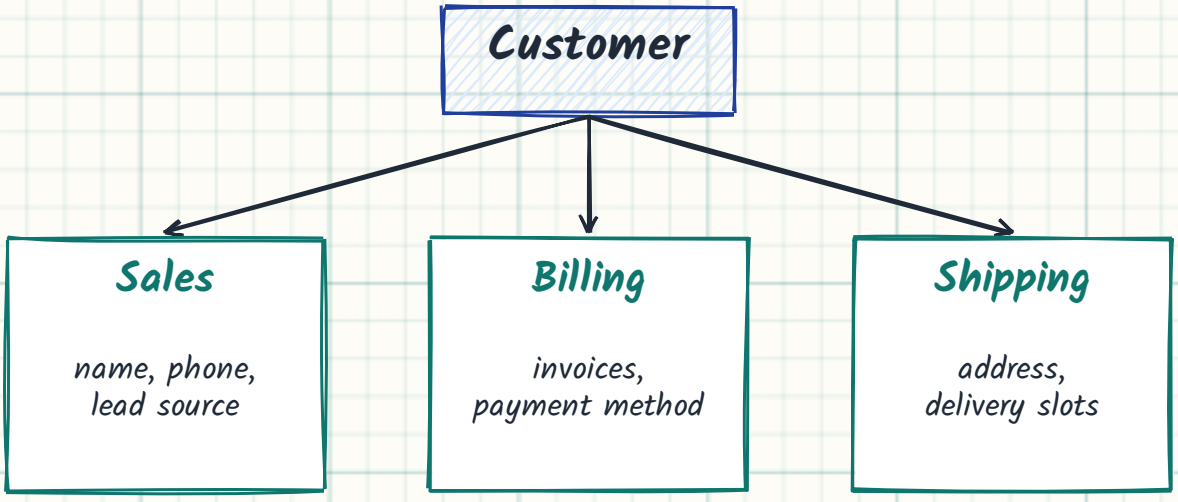
Lesson 3 · bounded contexts & aggregates for microservices

In this lesson → ① What is DDD ② Subdomains ③ Ubiquitous language ④ Bounded context ⑤ Context map ⑥ Entities & value objects ⑦ Aggregates ⑧ Contexts → services ⑨ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① What is Domain-Driven Design?

> **DDD** (Eric Evans, *Domain-Driven Design*, 2003) = design software around the business domain, building the model together with **domain experts**.



same word · 3 different meanings

Strategic design → the big boundaries
subdomains · ubiquitous language · bounded contexts · context map

Tactical design → the code inside a boundary
entities · value objects · aggregates · domain events

For microservices
Strategic design matters most: it tells you **where to draw service boundaries.**

② Subdomains



Type	Meaning	ShopNest	Strategy
Core	What makes the business win	Ordering, Pricing	Build in-house, best people
Supporting	Needed but not unique	Inventory, Shipping	Build simply or outsource
Generic	Already solved everywhere	Login, Email/SMS	Buy / use a ready-made tool

③ Ubiquitous language

> A **shared vocabulary** used by business people and developers, in meetings, documents and in the code.

✓ Business: "place an order" → code: `placeOrder()`

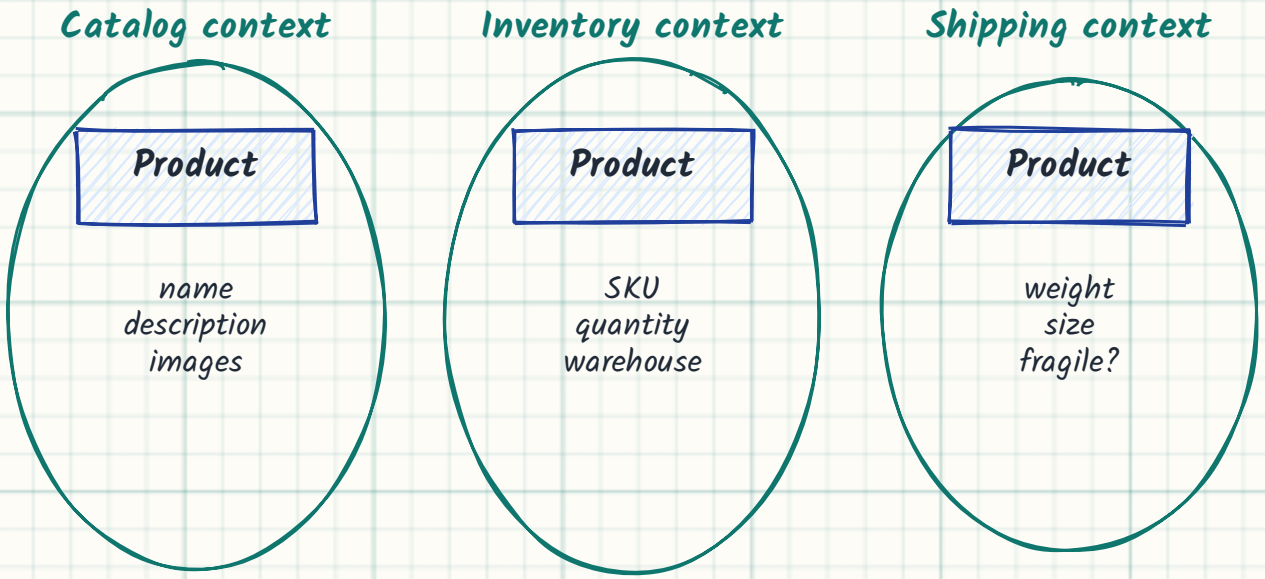
✓ Same term, same meaning, everywhere in one context

✗ `insertOrderRecord()`, `processData()`

✗ Developers inventing their own names

④ Bounded context

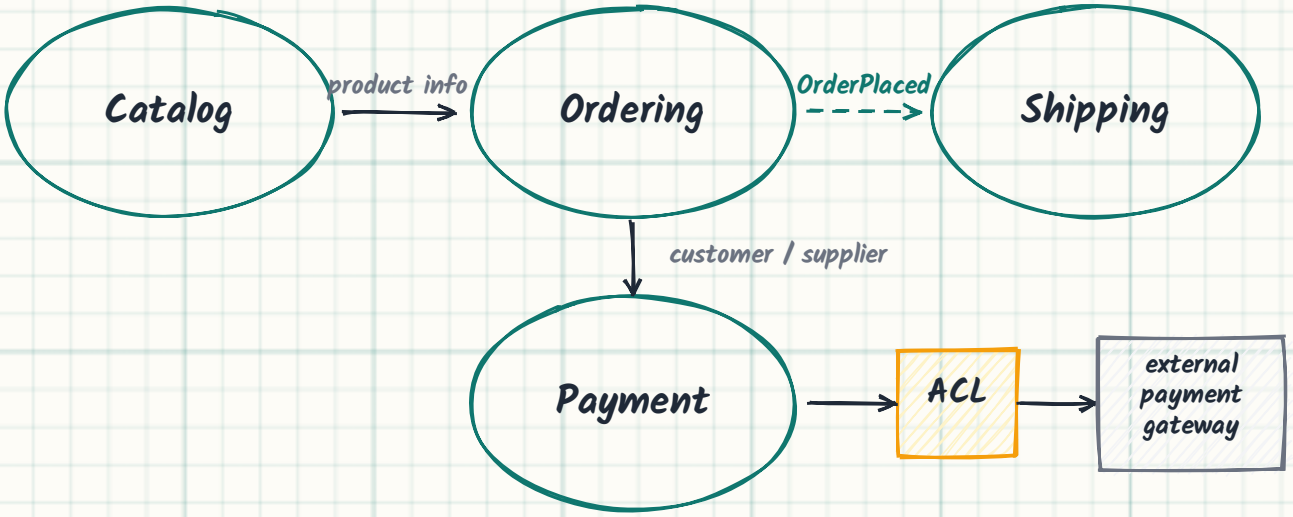
> **Bounded context** = a boundary inside which one model and one language are consistent. The same word may mean something else in another context.



Key insight
Don't build one giant **Product** or **Customer** class for everyone. Each context keeps **its own model** with only the fields it needs.

⑤ Context map

- → **Customer / Supplier**: downstream (Ordering) depends on upstream (Payment); they agree on the contract
- → **Published language / events**: Ordering publishes `OrderPlaced`, Shipping listens
- → **Anti-corruption layer (ACL)**: translates an external model (payment gateway) into yours
- → **Shared kernel**: two contexts share a small common model (use rarely)



⑥ Entities & value objects

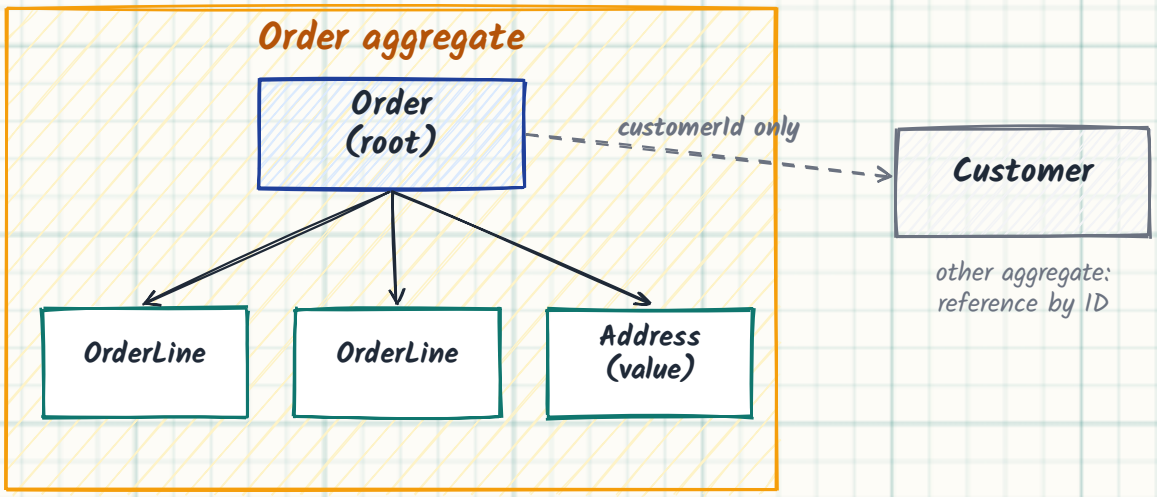
	Entity	Value object
Identity	Yes: Order #123	No
Equality	Same ID = same thing	Same values = same thing
Changes?	State changes over time	Immutable (create a new one)
ShopNest	Order, Customer, Product	Money, Address, Quantity

ShopNest
Money(500, INR) is a value object: two equal amounts are interchangeable. Order #123 is an entity: it stays the same order even when its status changes.

⑦ Aggregates

> **Aggregate** = a cluster of objects treated as one unit for data changes, with one entry point: the **aggregate root**.

1. Change the aggregate **only through its root**
2. **One transaction** changes **one aggregate**
3. Reference other aggregates **by ID**
4. Keep aggregates **small**

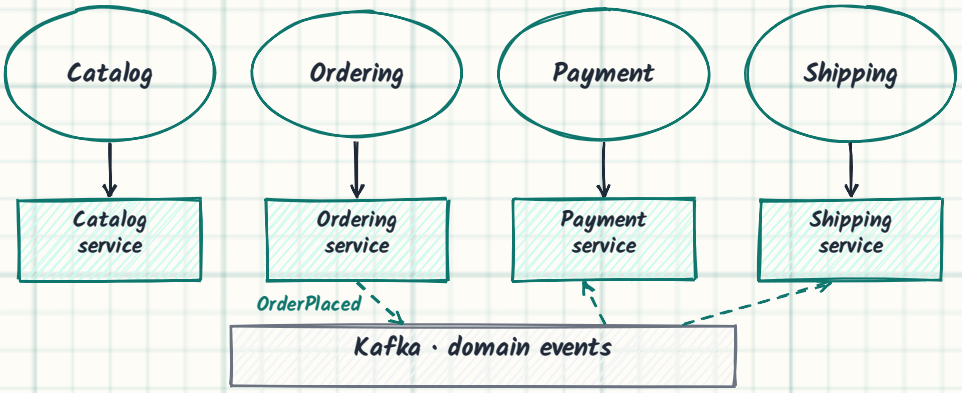


Order aggregate root in Java

```
public class Order {            // aggregate root
  private final OrderId id;
  private final CustomerId customerId;  // other aggregate: ID only
  private final List<OrderLine> lines = new ArrayList<>();
  private OrderStatus status = OrderStatus.NEW;

  public void addItem(ProductId p, int qty, Money price) {
    if (status != OrderStatus.NEW)
      throw new IllegalStateException("Order already placed");
    if (qty <= 0) throw new IllegalArgumentException("qty");
    lines.add(new OrderLine(p, qty, price));   // rule kept here
  }
}
```

⑧ From bounded contexts to microservices



→ **1 bounded context → 1 service** is the best starting point (never 1 service across 2 contexts)

→ Inside an aggregate → normal ACID transaction

→ Between aggregates / services → **domain events** + eventual consistency

Interview Corner

Q. What is a bounded context?

A. A boundary where one domain model and its ubiquitous language are consistent. The same word can mean different things in other contexts.

Q. What is an aggregate and an aggregate root?

A. A cluster of objects changed as one unit. All changes go through the root, which protects the business rules. One transaction per aggregate.

Q. Entity vs value object?

A. An entity has identity that survives state changes; a value object is defined by its values and is immutable.

Q. What is an anti-corruption layer?

A. A translation layer that keeps an external or legacy model from leaking into your bounded context.

Q. How does DDD help microservices?

A. Bounded contexts give service boundaries; aggregates define where strong consistency is needed; domain events connect services.

⚡ Revise in 60 seconds

- → **DDD**: Eric Evans, 2003
- → Core · supporting · generic subdomains
- → **Ubiquitous language** in the code
- → **Bounded context** → service boundary
- → Context map: customer/supplier, events, **ACL**
- → Entity = identity · value object = values
- → Small **aggregates**, change via root
- → Between services → **domain events**

Tip

Find boundaries in an **Event Storming** workshop (Alberto Brandolini): sticky notes for events, commands and aggregates.

Next lesson → MS-04

Breaking a monolith: **service boundaries & the Strangler Fig pattern.**

Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**